Session 9: Arrays — Notes & Code

C Language Study Notes & Code Guide

Key Concepts: Arrays

- **1D Array:** A list of elements of the same type, stored in contiguous memory. Index starts at **0** — a 7-element array has indices 0 to 6.
- **2D Array:** A table with rows and columns — `arr[rows][cols]`.
- **Traversal:** Visiting every element using loops. 1D needs one loop; 2D needs **nested loops** (outer = rows, inner = columns).
- Arrays in C are always passed to functions as **pointers** (pass-by-reference behavior), so changes persist and the size must be passed separately.

1 Task 1: `dailySteps` — 1D Array of 7 Step Counts

Notes: Declare with size 7, initialize with sample values using `{ }` braces. Traverse with a `for` loop from index `0` to `6`. Print with day labels to make output meaningful (`i + 1` = day number).

```c
#include <stdio.h>

int main() {
    int dailySteps[7] = {8500, 10200, 7450, 12000, 9300, 6800, 11000};

    printf("Weekly Step Count:\n");
    for (int i = 0; i < 7; i++) {
        printf("Day %d: %d steps\n", i + 1, dailySteps[i]);
    }
    return 0;
}
```

Sample Output:

```
Weekly Step Count:
Day 1: 8500 steps
Day 2: 10200 steps
Day 3: 7450 steps
Day 4: 12000 steps
Day 5: 9300 steps
Day 6: 6800 steps
Day 7: 11000 steps
```

2 Task 2: `playlistRatings` — 2D Array (3 Playlists × 5 Days)

Notes: 2D array declared as `playlistRatings[3][5]` — 3 rows (playlists), 5 columns (days). Print **only the second playlist** = row index **1** — traverse just that row with a single loop (no nested loop needed when accessing one row).

```c
#include <stdio.h>

int main() {
    // rows = playlists, columns = days
    int playlistRatings[3][5] = {
        {4, 5, 3, 4, 5},     // Playlist 1
        {3, 4, 4, 5, 2},     // Playlist 2
        {5, 5, 4, 3, 4}      // Playlist 3
    };

    printf("Ratings for Playlist 2 (over 5 days):\n");
    for (int day = 0; day < 5; day++) {
        printf("Day %d: %d stars\n", day + 1, playlistRatings[1][day]);
    }
    return 0;
}
```

Sample Output:

```
Ratings for Playlist 2 (over 5 days):
Day 1: 3 stars
Day 2: 4 stars
Day 3: 4 stars
Day 4: 5 stars
Day 5: 2 stars
```

3 Task 3: Function to Calculate Average Zomato Spend

Notes: Function takes the array and its size (size must be passed because arrays decay to pointers inside functions). Sum all elements with a loop, then divide by length. Use `float`/`double` for the average to keep decimal accuracy (integer division would truncate).

```c
#include <stdio.h>

// Function: takes 1D array of 7 order amounts, returns average
float averageSpend(int orders[], int length) {
    int sum = 0;
    for (int i = 0; i < length; i++) {
        sum += orders[i];
    }
    return (float)sum / length;   // cast to float for decimal average
}

int main() {
    int zomatoOrders[7] = {250, 340, 180, 420, 299, 510, 365};

    float avg = averageSpend(zomatoOrders, 7);
    printf("Total weekly spend : Rs.%d\n", 250+340+180+420+299+510+365);
    printf("Average daily spend: Rs.%.2f\n", avg);
    return 0;
}
```

Sample Output:

```
Total weekly spend : Rs.2364
Average daily spend: Rs.337.71
```

4 Task 4: `cricketScores` — Highest Score from Each Match

Notes: Each row = one IPL match, each column = runs by one team (e.g., 2 teams per match). For every row, run an inner loop that finds the maximum — start with `max = row[0]` (first element), then compare with the rest. This is the classic "row-wise maximum" pattern.

```c
#include <stdio.h>

int main() {
    // 3 matches, each with runs by 2 teams
    int cricketScores[3][2] = {
        {198, 174},    // Match 1
        {156, 201},    // Match 2
        {189, 185}     // Match 3
    };
    int matches = 3, teams = 2;

    printf("Highest score in each match:\n");
    for (int i = 0; i < matches; i++) {
        int max = cricketScores[i][0];   // assume first team is highest
        for (int j = 1; j < teams; j++) {
            if (cricketScores[i][j] > max) {
                max = cricketScores[i][j];   // found a bigger score
```

```
            }
        }
        printf("Match %d: %d runs\n", i + 1, max);
    }
    return 0;
}
```

Sample Output:

```
Highest score in each match:
Match 1: 198 runs
Match 2: 201 runs
Match 3: 189 runs
```

5 Quick Revision Summary

Task	Array Type	Key Logic
1	1D (7 elements)	Simple `for` loop traversal, index 0 to 6
2	2D `[3][5]`	Access one row via `arr[1][day]`
3	1D + function	Sum in loop → divide by length → `float` average
4	2D `[3][2]`	Row-wise maximum (nested loop + `if > max`)

Golden Rules of Arrays in C:

- Indexing starts at **0**, so the last index is `size - 1`.
- 2D arrays: **outer loop = rows, inner loop = columns**.
- When passing arrays to functions, always pass the **size as a separate parameter** — the function cannot know it on its own.